Exercise 3

Task 1

1.1

Implementation is in `src/task1/`, needs zenoh-cpp installed to be compiled. To compile run `cmake -B build -S .` and cmake `--build build`.

1.2

From NodeA's perspective:

```
[PEERS] nodeA sees 2 peer(s): mesh/nodes/nodeB mesh/nodes/nodeC
[LEAVE] mesh/nodes/nodeB
[PEERS] nodeA sees 1 peer(s): mesh/nodes/nodeC
[PEERS] nodeA sees 1 peer(s): mesh/nodes/nodeC
[JOIN] mesh/nodes/nodeB
[PEERS] nodeA sees 2 peer(s): mesh/nodes/nodeB mesh/nodes/nodeC
```

From NodeB's perspective:

```
[PEERS] nodeB sees 2 peer(s): mesh/nodes/nodeA mesh/nodes/nodeC
^C
[odin task1]$ ./build/mesh_node nodeB
[JOIN] mesh/nodes/nodeA
Node nodeB is online.
Listening for other nodes. Ctrl+C to exit.
[JOIN] mesh/nodes/nodeC
[PEERS] nodeB sees 2 peer(s): mesh/nodes/nodeA mesh/nodes/nodeC
```

- How does a node learn that another node is available? When a new node starts, it declares a liveliness token, every subscriber of the liveliness in the current zenoh session then gets a `PUT` message, i.e. an update that a new node joined the session. This all happens asynchronously, which is why there is a need for synchronisation in the handler of the subscriber and the data that gets accessed by the sequential part of the program.
- What happens when Node B leaves, and what happens when it joins again? When a node leaves, it sends a `DELETE` message to every subscriber of the liveliness session. So every other node can handle this and save that the node is no longer available. If the node joins again it is essentially the same as in the first point
- Your discovery mechanism tells you a node is reachable. Can you trust that a discovered node is who it claims to be, or that its messages are truthful? No not necessarily a node could potentially give a false `PUT` and `DELETE` message which is not necessarily the actual node, if it knows the id

Task 2

2.1

Implementation is in `src/task2/`.

2.2

- What is the main difference between static configuration and dynamic discovery? In dynamic discovery the connections don't have to be fixed in advance, but can be made dynamically, this is much more flexible. For static configuration, one has too manually manage all connections and the topology, and needs to restart the application if there was a change.
- Why is the publish/subscribe pattern useful for communication between distributed processes? The pattern is usefull because it is handled automatically and asynchronously without the need of constantly polling to get information/to know if information is there.
- Is peer discovery sufficient to make an application fault-tolerant? Explain briefly. No, for example if the first discovery message is already lost, a connection will never be established.

Task 3

3.1

Implementation of the 3 Node problem is in `src/task3/byzantine3.cpp`.

3.2

For the all loyal test it behaves as expected. The leutenants receive an order from the commander and the other leutenant and they match. If the commander is a traitor, every leutenant will have a different order from the commander and the other leutenant. The leutenant can't distinguish if the other leutenant or the commander is a liar, because if the commander was the traitor then the other loyal leutenant will have a different order from the commander and if the commander is loyal and the other leutenant the traitor, the traitor will receive the correct order but send a different one. In both cases the 2 commands differ.

3.3

Code is in `src/task3/byzantine4.cpp`

3.4

- Why does majority voting succeed with three lieutenants but fail with only two? Because there is no majority for only 2 leutenants, if the two sources disagree in contrast to 3.

- In your implementation the commander is a designated, central node. Tasks 1–2, by contrast, built a fully decentralized mesh without a coordinator. What does the commander provide that makes agreement possible with four nodes, and why does the problem become harder when all nodes are equal? The commander is a single starting point, so every leutenant is working on the same decision. With multiple starting points one first has to define what the correct outcome would be.
- Your four-node implementation guarantees agreement, but can it identify which node is the traitor? Explain what a loyal node can and cannot observe. If the commander is the traiter, can only be determined by a node, if the action from the commander deviates from both orders of the other leutenants. So he could potentially broadcast that the commander is treacherous, this message would not mean much to the others, since he could be lying and the other nodes have no way to check. If the Leutenant is the traitor there is no way to determine who the traitor is by the other nodes.
- By default, nodes often use automatic multicast scouting to discover each other on a local network. In restricted networks (or wide-area networks) where multicast is blocked. How could nodes communicate in such cases in practice? Investigate whether the middleware you selected offers such a feature. If multicast scouting is not available, then there needs to be a central node or router which is always statically reachable/known to all nodes. If a node registers, the central node saves this and sends the newly node the previously saved registrations, if a node derigesters, it can then send this information to all the other registered nodes. Zenoh has this functionality with zenohd.